Programación Concurrente y Distribuida

Tema 9. Exclusión mutua en Sistemas Distribuidos

Bibliografía de apoyo:
M.C. Garrido; J.T. Palma.
Programación Concurrente. Capítulo 7.3
Diego Marín. Colección Texto Guía. 2001.

Contexto

- Sistemas distribuidos:
  - Ausencia de memoria compartida
  - Ausencia de reloj común
  - Comunicaciones no fiables entre nodos
  - Estado global no observable
- Problemas
  - Garantizar Exclusión Mutua en el acceso a recursos compartidos mediante algoritmos distribuidos
  - Pasarse información (y/o tenerla actualizada en todos los nodos) y coordinarse.

Requisitos de la solución

- Requisitos deseables para un algoritmo distribuido
  - Igualdad en cuanto a la cantidad de información que debe tener cada nodo
  - Toma de decisiones local
  - Igualdad de esfuerzo para tomar decisiones
  - Tolerancia a fallos
- Propiedades importantes de un algoritmo distribuido
  - Simetría: cantidad y tipo de información por nodo
  - Coste: número de mensajes necesarios

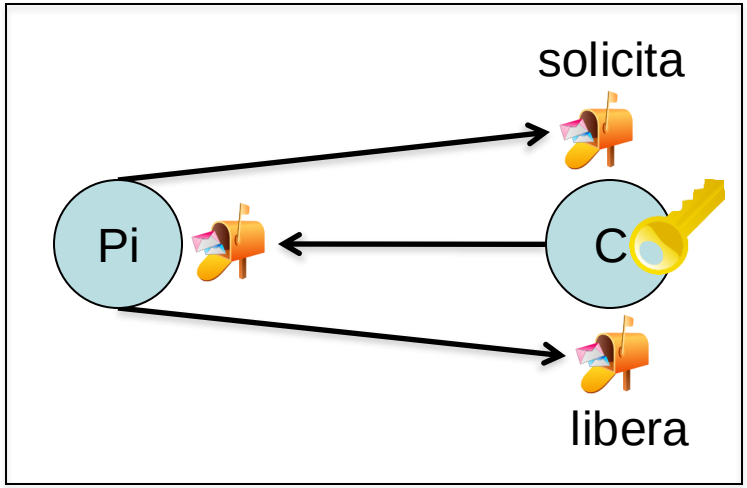
Exclusión mutua en Sistemas Distribuidos (SD)

- Enfoque centralizado
- Enfoque basado en paso de testigo
- Enfoque distribuido

Enfoque centralizado

Un proceso actúa de coordinador

1. Antes de entrar a una sección crítica se debe pedir permiso al coordinador enviándole un mensaje de solicitud
2. La entrada a la sección crítica estará condicionada a la recepción del mensaje de confirmación
3. Al salir de una sección crítica se debe informar al coordinador con un mensaje de liberación



Enfoque centralizado con Buzones

Coordinador

```
boolean libre[N] = {true,..., true};
Cola espera[N] = {new Cola(), ..., new Cola()};
while (true) {
  select {
      receive(solicitud, peticion);
      if (libre[peticion.sc()]) {
       libre[peticion.sc()] = false;
       send(peticion.br(), ok);
      } else {
       espera[peticion.sc()].add(peticion);
      }
    or
      receive(libera, sc);
      libre[sc] = true;
      if (!espera[sc].empty()) {
       libre[sc] = false;
       peticion = espera[sc].pop();
       send(peticion.br(), ok);
      }
  }
}
```

MailBox solicitud;
MailBox libera;

Pi

```
MailBox br = new MailBox();
while (true) {
  Peticion peticion =
    new Peticion(ID, br);
  send(solicita, peticion);
  receive(br, ok);
  // Sección Crítica ID
  send(libera, ID);
}
```

Análisis del enfoque centralizado

- Garantiza la exclusión mutua
- Es justo y la espera es limitada
- Tiene un coste de 3 mensajes por uso de sección crítica
- Es **asimétrico** ya que el coordinador tiene más información que los demás procesos
- Es poco tolerante a fallos ya que si cae el coordinador todo el sistema deja de funcionar
- El coordinador es un cuello de botella

Enfoque basado en paso de testigo

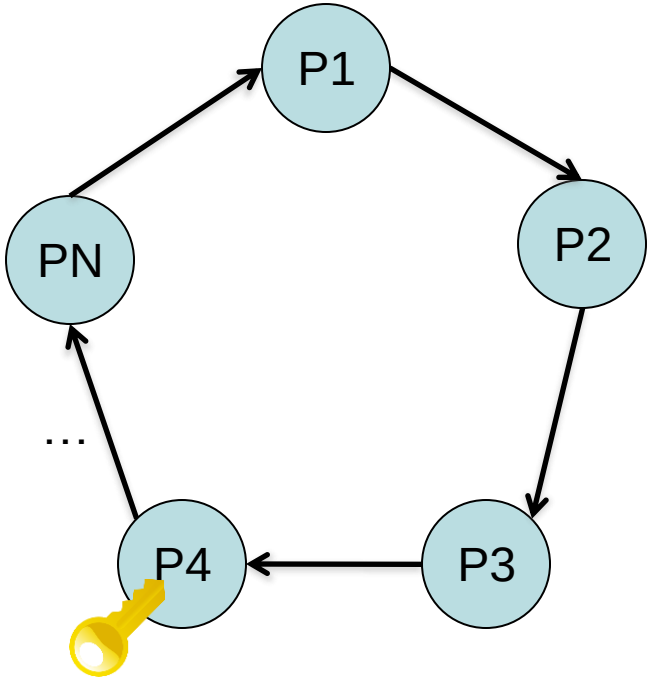
- Idea:

  Se hace circular entre los procesos del sistema un mensaje especial y único, llamado testigo, que faculta al poseedor para acceder a la sección crítica

- Alternativas
  - Estructurado en anillo: Algoritmo de Le Lann
  - No estructurado: Algoritmo de Chandy

Algoritmo de Le Lann

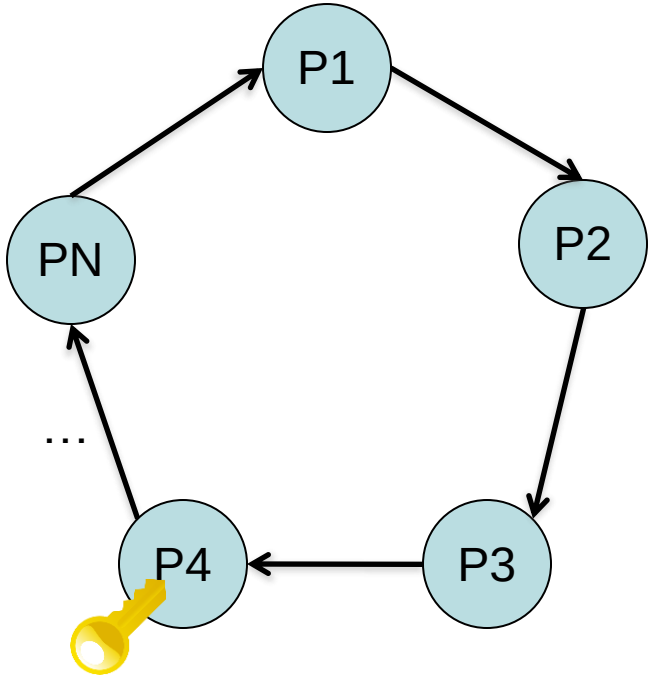
- Los procesos deben formar un anillo lógico
- Al recibir el testigo se puede entrar en la sección crítica conservándolo mientras tanto
- Al salir de la sección crítica se pasa el testigo al siguiente proceso del anillo lógico
- Si se recibe el testigo pero no se necesita se pasa inmediatamente al siguiente proceso del anillo lógico

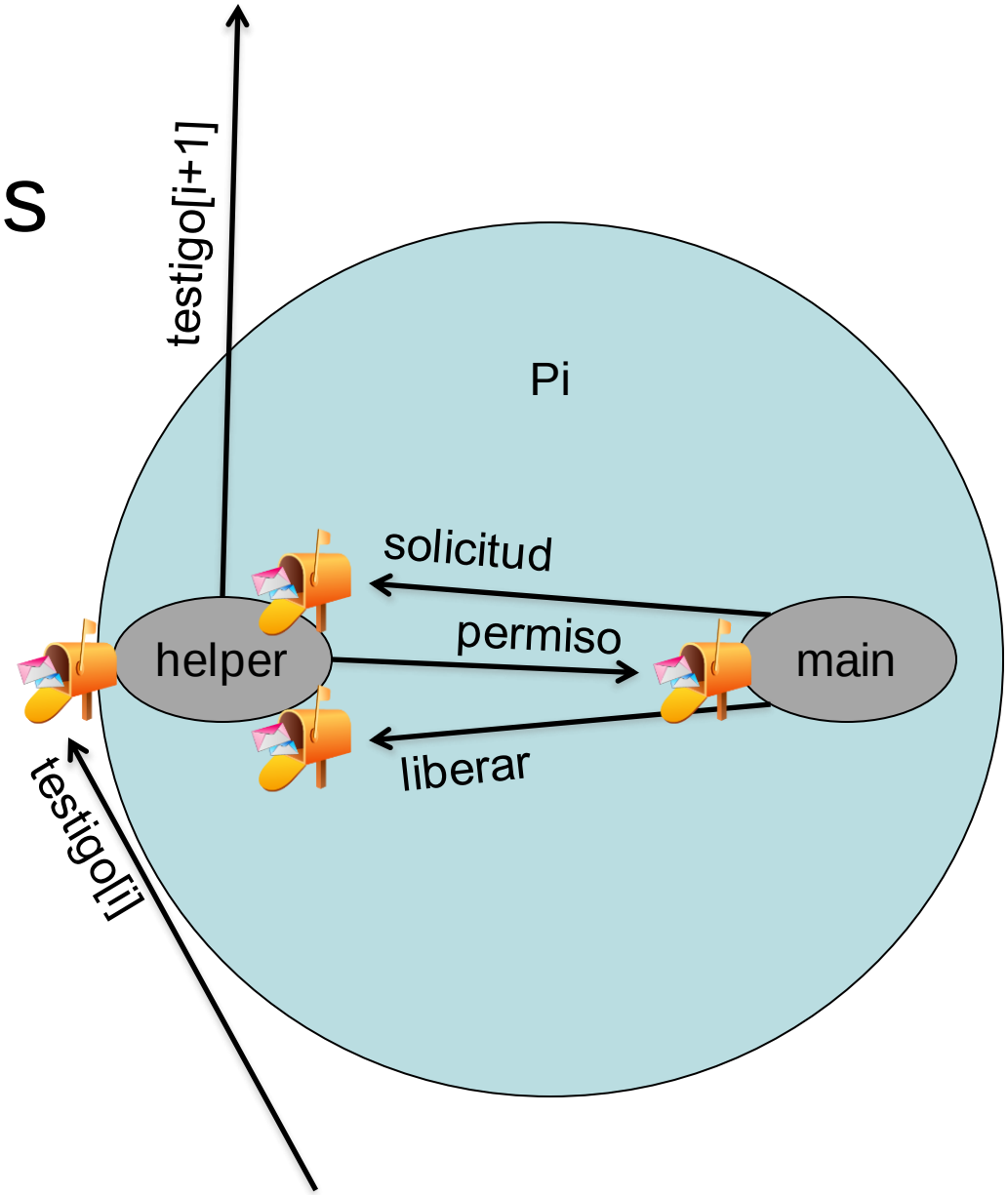


Pi

```
...
Esperar testigo de Pi-1
// Sección Crítica
Pasar testigo a Pi+1
...
```

Algoritmo de Le Lann con Buzones





El helper es fundamental para permitir que el testigo siga circulando cuando los procesos no lo necesitan.

Le Lann con buzones

- int N // Número de procesos
- Mailbox solicitud; // buzon local para solicitar entrar en S.C.
- Mailbox permiso; // buzón local para recibir permiso
- Mailbox liberar; // buzón local para informe de que ha salido de su S.C.
- Mailbox testigo[N]; // buzón para circular el testigo entre los procesos del anillo lógico

Pi: helper thread

```
while (true) {
  receive(testigo[i], msg);
  select {
      receive(solicitud, msg);
      send(permiso, msg);
      receive(liberar, msg);
      send(testigo[i+1], msg);
    else
      send(testigo[i+1], msg);
  }
}
```

Pi: main thread

```
...
// Esperar testigo de Pi-1
send(solicitud, msg);
receive(permiso, msg);
// Sección Crítica
// Pasar testigo a Pi+1
send(liberar, msg);
...
```

Análisis del algoritmo de Le Lann

- Exclusión mutua garantizada por la existencia de un único testigo
- Falla el progreso en la ejecución (espera a que llegue el testigo)
- Coste: desde un mensaje por proceso si todos quieren entrar hasta infinitos si nadie quiere entrar
- Es simétrico
- Es poco tolerante a fallos ya que la pérdida del testigo o fallo de un proceso hace caer al sistema

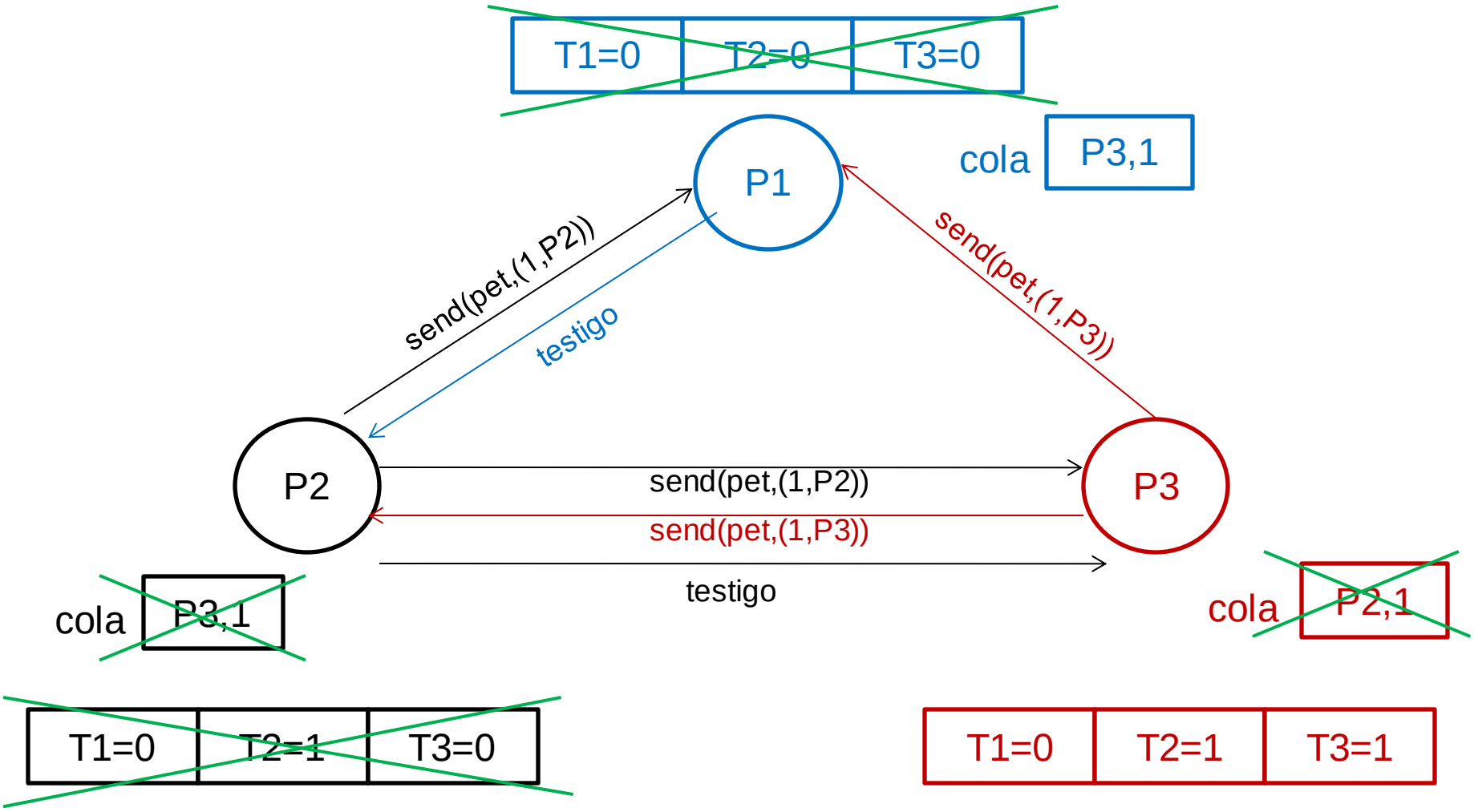
Algoritmo de Chandy

- Ideas clave
  - No supondremos ninguna orden lógico entre los procesos participantes.
  - Evitaremos enviar mensajes innecesarios cuando nadie quiera el testigo.
  - Cuando alguien pida el testigo el proceso que lo tenga, si ya no lo necesita, lo cederá.

Algoritmo de Chandy

- Algoritmo
  - El testigo es un array **T={T1, T2, …, TN}** en el que **Ti** indica el número de veces que Pi entró en su S.C.
  - Cuando se quiere entrar en la S.C. se envía una solicitud al resto de procesos indicando que se quiere entrar por **n**-esima vez
  - Cuando el proceso **i** recibe el testigo que había solicitado actualiza **Ti** y entra en su S.C.
  - Al salir de la S.C. se examina la cola de peticiones y se envía el testigo al proceso que primero lo hubiera pedido descartando aquellas cuyo valor sea inferior al valor de su correspondiente **Ti**

Simulación de Chandy



Pi: helper thread

```
int vez = 0; boolean enSC = false; int []T = null; Cola cola = new Cola();
while (true) {
  select {
      receive(solicitud, msg);
      vez++;
      if (T == null) {
        for ( int j = 0 ; j < N ; j++ )
          if (j!=id) send(peticion[j], new Peticion(id, vez)); // id => Pi
        receive(testigo[id], T);
      }
      T[id] = vez;      enSC = true;
      send(permiso, ok);
    or
      receive(liberar, msg);
      enSC = false;
      boolean encontrado = false;
      while (!c.vacia() && !encontrado) {
         Peticion p = c.pop();
         if (p.vez() > T[p.id()]) {
           send(testigo[p.id()], T);
           T = null;   encontrado = true;
         }
      }
    or
      receive(peticion[id], p);
      if (T != null && !enSC) {
         send(testigo[p.id()], T);
         T = null;
      } else {  cola.push(p); }
  }
}
```

Chandy con buzones

Pi: main thread

```
...
// Esperar testigo
send(solicitud, msg);
receive(permiso, ok);
// Sección Crítica
send(liberar, msg);
// Pasar testigo
...
```

Análisis del algoritmo de Chandy

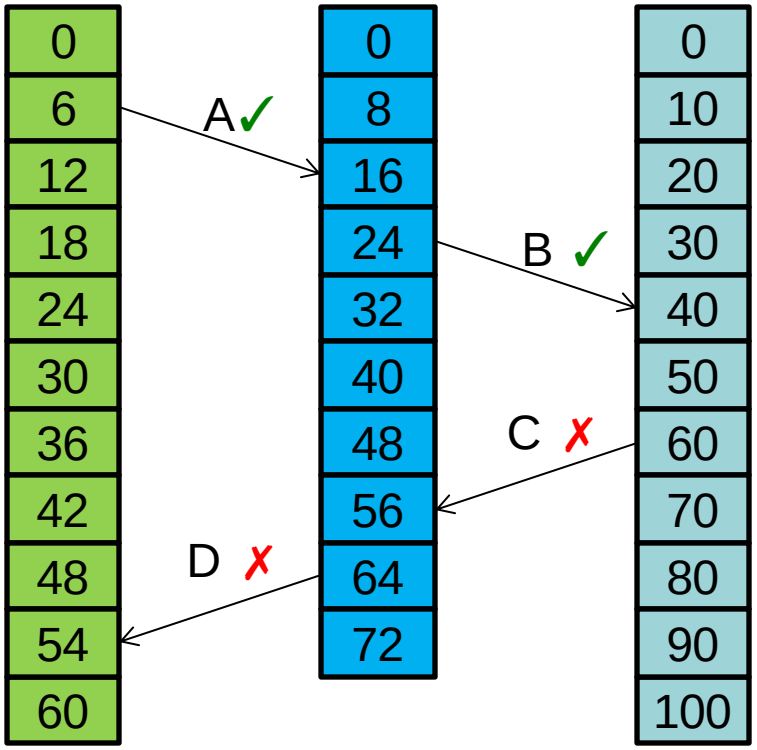
- Garantiza la exclusión mutua gracias a la existencia de un único testigo
- Coste: n mensajes para acceder a la sección crítica
- Es simétrico
- El sistema cae sólo si el proceso que tiene el testigo falla en ese momento

Enfoque distribuido (ordenar sucesos en el tiempo)

- En sistema centralizado la existencia de un reloj central permite ordenar los sucesos.
- En un sistema distribuido cada nodo del sistema tiene un reloj que no tiene porqué estar sincronizado con los demás
- La ordenación de sucesos en un sistema de este tipo será más compleja.
- En 1978 Lamport ideó un mecanismo para ordenar sucesos en un sistema distribuido.
  - Ordenación de sucesos y relojes lógicos de Lamport (1978)
- Algoritmo de Ricart y Agrawala (E-M con relojes)

Sucesos en Sistemas Distribuidos

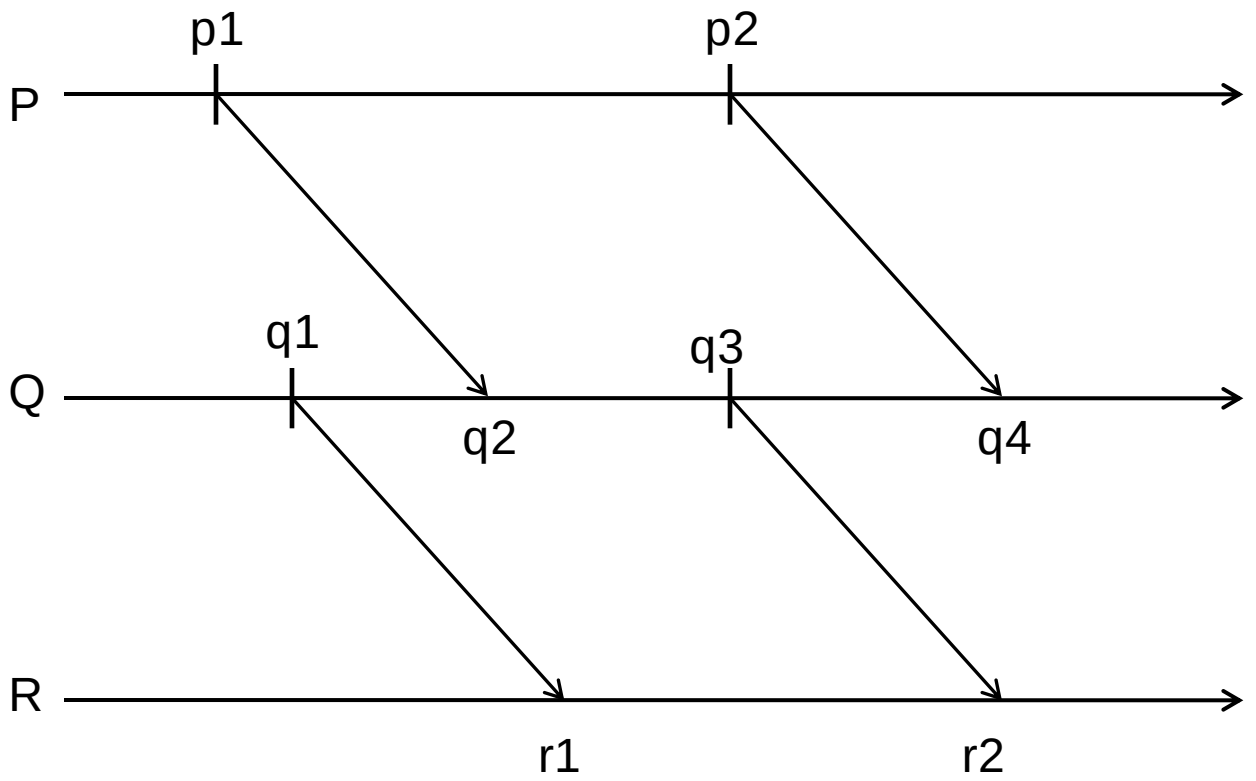
- La utilidad del paso de mensajes reside en que establece una relación temporal de eventos ya que la recepción siempre se produce después del envío.
- Cada nodo del Sistema Distribuido tiene su propio reloj por lo que es necesario poder establecer el orden en que los distintos envíos y recepciones (eventos) que suceden en el sistema.



Ordenación de sucesos

- Definimos la relación “sucedió antes” como sigue:
  - Si **a** y **b** son sucesos del mismo proceso y **a** ocurre antes que **b** diremos que **a->b** (**a** sucedió antes que **b**)
  - Si **a** representa el envío de un mensaje y **b** su recepción (aunque sea en otro proceso) **a->b**
  - Si **a->b** y **b->c** entonces **a->c**
- Esta relación define un orden parcial entre los sucesos del sistema. Cuando dos sucesos no se puedan relacionar diremos que son concurrentes

Ejemplo de relación entre sucesos



p1->p2
q1->q2->q3->q4
r1->r2

p1->q2
p2->q4
q1->r1
q3->r2

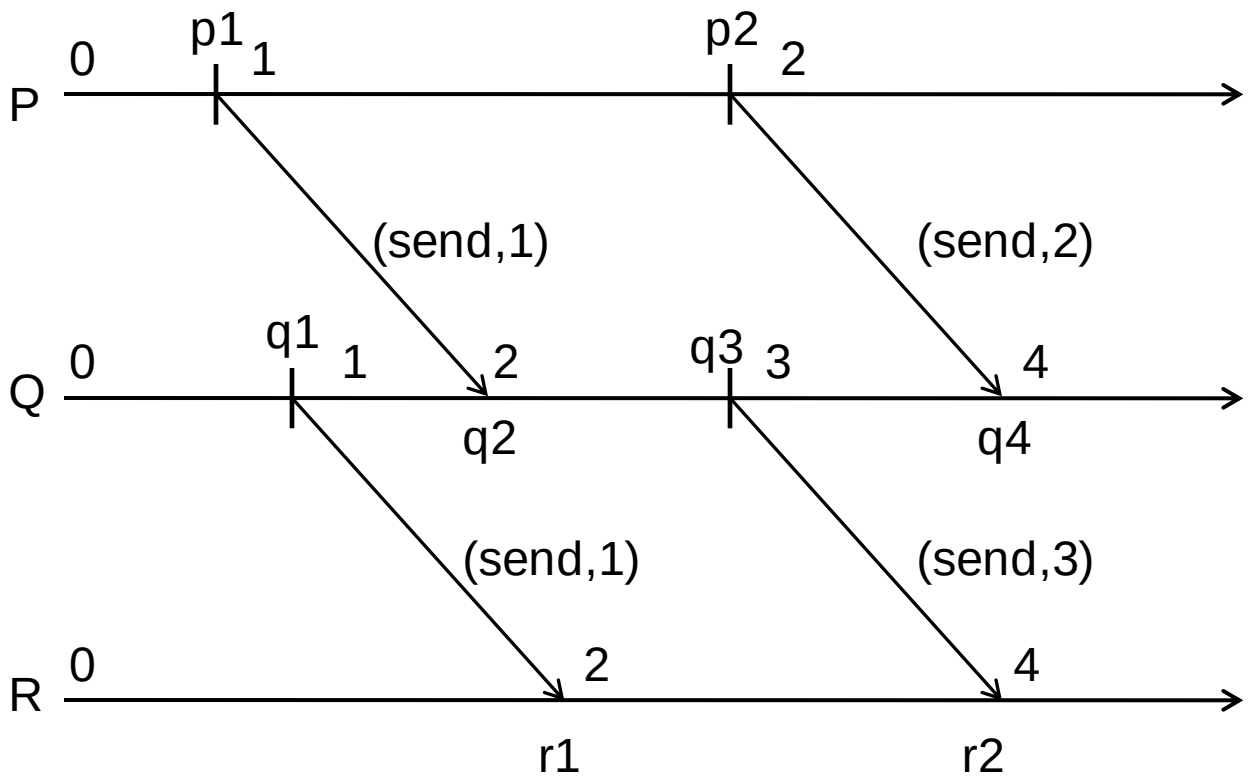
p1->r2
q1 ->r2

Dos sucesos son concurrentes si no existe un camino entre ellos.
Por ejemplo (p1,q1)

Relojes lógicos de Lamport

- Los relojes lógicos de Lamport permiten ordenar sucesos en un S.D. mediante el uso de contadores de sucesos internos a cada proceso que son incluidos en los mensajes
- Implementación
  - Cada proceso tiene su propio reloj lógico (contador Ri) inicializado a 0
  - Cada vez que se va a enviar un mensaje msg se incrementa Ri y se adjunta en el mismo (msg.timestamp)
  - Cada vez que se recibe un mensaje msg, Ri se actualiza así: Ri = 1 + max(Ri, msg.timestamp)

Relación entre sucesos con Lamport

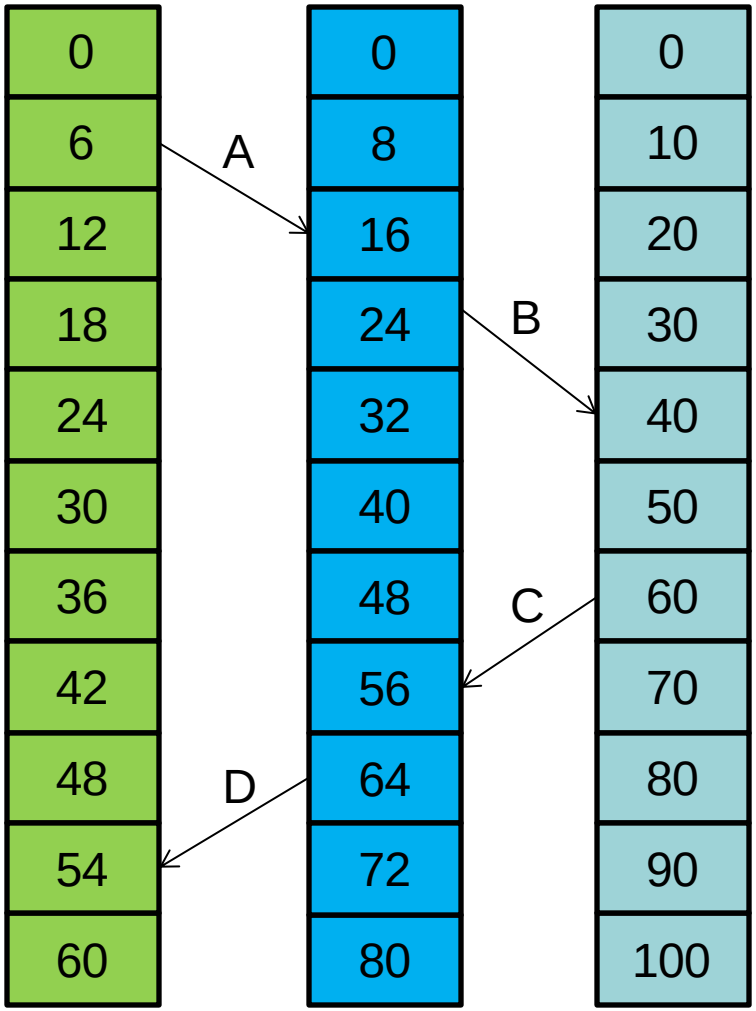


p1->p2
q1->q2->q3->q4
r1->r2

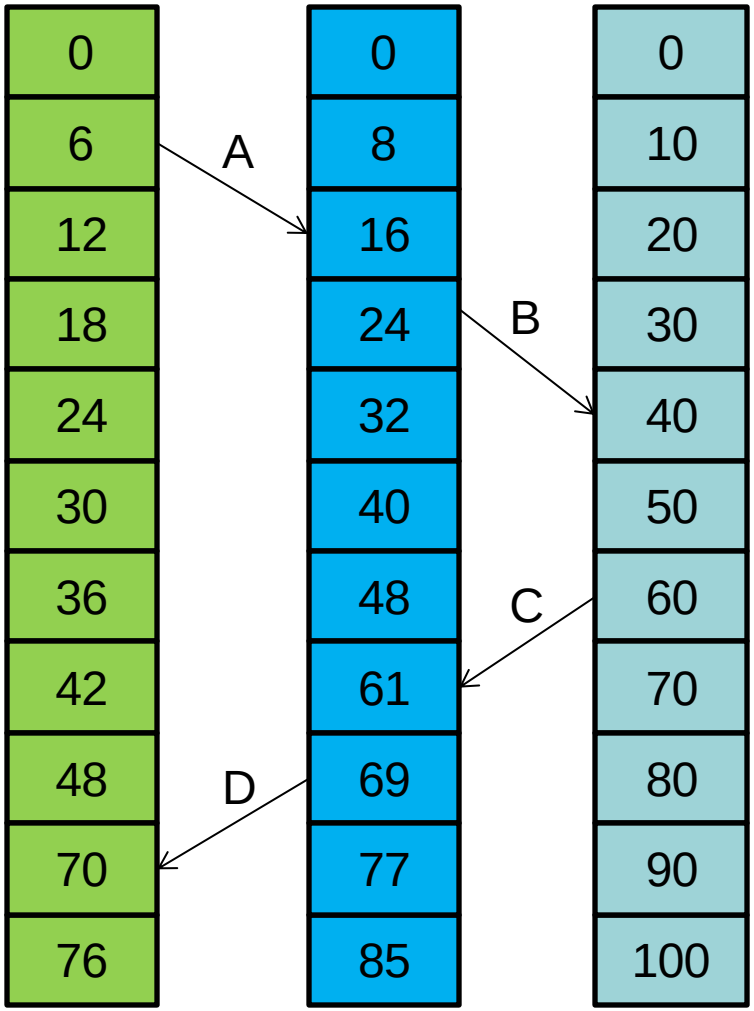
p1->q2
p2->q4
q1->r1
q3->r2

p1->r2
q1 ->r2

Ajuste de relojes con Lamport



Sin Lamport



Con Lamport

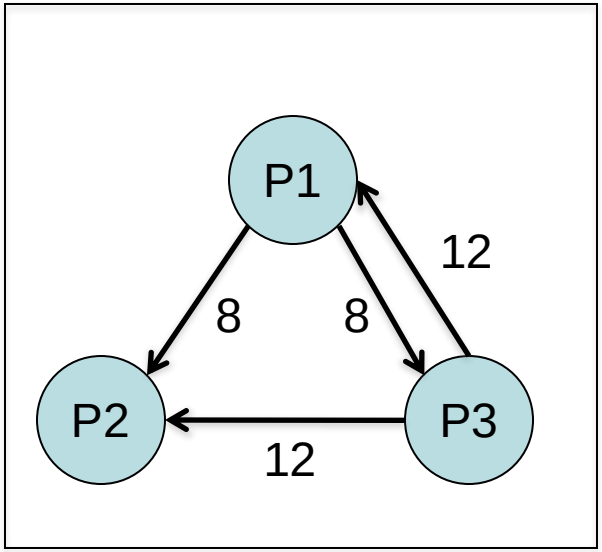
Algoritmo de Ricart y Agrawala

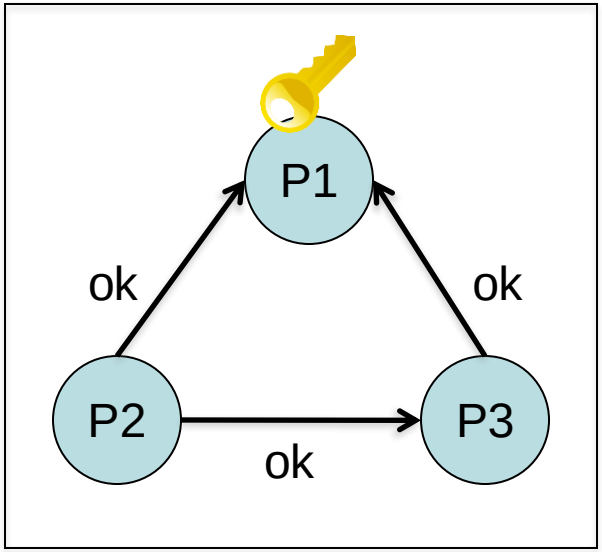
- Idea:
  - Cada proceso tiene una cola de peticiones y un reloj lógico de Lamport.
  - Antes de entrar en su S.C. pide permiso a los demás.
  - Entrará cuando tenga permiso de todos los demás.
  - Si dos procesos quieren entrar primero lo hace el que lo hubiera pedido antes según el tiempo lógico. En caso de igualdad de tiempos decide el id del proceso.

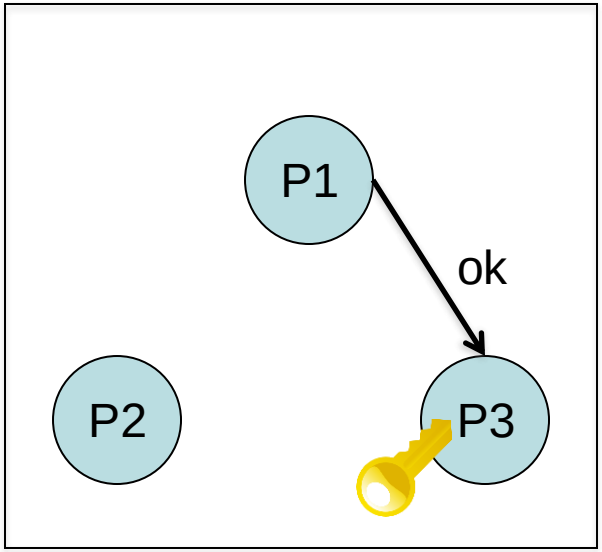
Algoritmo de Ricart y Agrawala

1. Cuando Pi quiere entrar a la S.C. envía un mensaje de petición (con timestamp) al resto de procesos y espera recibir confirmaciones
2. Pi entra en su S.C. después de recibir confirmación de todos
3. Cuando Pi sale de su S.C. envía confirmaciones pendientes de contestar
4. Cuando se recibe una petición se envía respuesta (con timestamp) solo si
   1. El proceso no está en S.C. ni interesado en adquirirla.
   2. Estando interesado, su propia solicitud es posterior al mensaje recibido o en caso de igualdad si id es mayor
5. Si no se da ninguno de estos dos casos la petición es almacenada en la cola

Simulación de Ricart y Agrawala







Pi: helper thread

```
boolean enSC = false;  boolean intentando = false;
int T = 0;  int Tsolicitud = 0;
int concedidos = 0;
Cola Q = new Cola();
while (true) {
  select {
    receive(solicitud, msg);
    intentando = true;
    T++;
    Tsolicitud = T;
    // petición al resto
    for ( int j = 0 ; j < N ; j++ ) {
      if (j != i)
        send(peticion[j], new Msg(id, T));
    }
  or
    receive(respuesta[id], msg);
    T = 1+max(T, msg.T);
    concedidos++;
    if (concedidos == n-1) {
      enSC = true;
      intentando = false;
      send(concesión, ok);
    }
...
```

Ricart y Agrawala

Pi: main thread

```
...
// Esperar testigo
send(solicitud, msg);
receive(concesión, ok);
// Sección Crítica
send(liberar, msg);
// Pasar testigo
...
```

Ricart y Agrawala

Pi: helper thread

```
  or
    receive(peticion[id], msg);
    T = 1+max(T, msg.T);
    if (enSC) Q.push(msg);
    else {
      if (!intentando) {
        T++;
        send(respuesta[msg.id], new Msg(id, T));
      } else {
        if (Tsolicitud > msg.T || (Tsolicitud == msg.T && id > msg.id)) {
          T++;
          send(respuesta[msg.id], new Msg(id, T));
        } else {
          Q.push(msg);
        }
      }
    }
  or
    receive(liberar, msg);
    enSC = false;
    concedidos = 0;
    while (!empty(Q)) {
         msg = Q.pop();
         T++;
         send(respuesta[msg.id], new Msg(id, T));
    }
  }
}
```

Análisis de Ricart y Agrawala

- Garantiza Exclusión mutua
- Es justo y no impide el progreso en la ejecución
- Es simétrico
- Es poco robusto ya que la caída de cualquier proceso impide a los demás entrar en su S.C.
- Coste: 2(N-1) por acceso a la S.C.